

# Learning Agent Execution for KV-Cache Management in Agentic Serving

Research Full

Rui Zhang<sup>1</sup>, Chaeun Kim<sup>1</sup>, Shaoting Feng<sup>2</sup>, Kuntai Du<sup>3</sup>, Yuhan Liu<sup>3</sup>, Yi Zhong<sup>1</sup>, Cheng-Wei Ching<sup>1</sup>,  
Junchen Jiang<sup>3</sup>, Liting Hu<sup>1</sup>

<sup>1</sup>University of California, Santa Cruz <sup>2</sup>University of Washington <sup>3</sup>University of Chicago

## Abstract

Multi-agent LLM systems have emerged as an important deployment paradigm for AI services, where each user request is decomposed into a sequence of specialized agents. Across these workflows, every agent repeatedly executes a fixed context consisting of system prompts, tool definitions, and few-shot examples, creating substantial opportunities for KV-cache reuse. Existing LLM serving systems, however, manage KV-cache reactively using prefix caching and recency-based replacement, causing reusable agent contexts to be evicted before their next invocation and forcing repeated recomputation. We present **CACHESCOUT**, an agent-aware KV-cache runtime layer for multi-agent LLM serving. The key insight is that future KV-cache reuse is governed by agent execution semantics rather than cache recency alone. **CACHESCOUT** captures these semantics by learning agent execution transitions online, without requiring predefined workflow graphs or offline training, and uses the learned execution model to guide both cache eviction and proactive prefetching while leaving the serving critical path unchanged. We implement **CACHESCOUT** on top of vLLM. Across representative real-world multi-agent workloads, **CACHESCOUT** improves KV-cache hit rate by 10–18 percentage points, reduces mean TTFT by 18–45%, lowers mean per-turn latency by 29–38%, and increases peak throughput by up to 57%. These benefits also generalize to larger models, reducing TTFT by up to 54% while sustaining 37% higher throughput.

## Keywords

LLM serving, multi-agent systems, KV cache, cache management, prefix caching, machine learning systems

## ACM Reference Format:

Rui Zhang<sup>1</sup>, Chaeun Kim<sup>1</sup>, Shaoting Feng<sup>2</sup>, Kuntai Du<sup>3</sup>, Yuhan Liu<sup>3</sup>, Yi Zhong<sup>1</sup>, Cheng-Wei Ching<sup>1</sup>, Junchen Jiang<sup>3</sup>, Liting Hu<sup>1</sup>. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving: Research Full. In . ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## 1 Introduction

Multi-agent LLM systems have rapidly become the dominant deployment paradigm for production AI services, powering applications such as software engineering assistants [15, 42, 50], enterprise knowledge assistants [10, 19, 36], and deep research agents [8, 13, 49]. Instead of relying on a single LLM call, these applications decompose each user request into subtasks handled by a sequence of specialized agents that collaborate to complete complex tasks. A planner coordinates the workflow, domain-specific agents perform

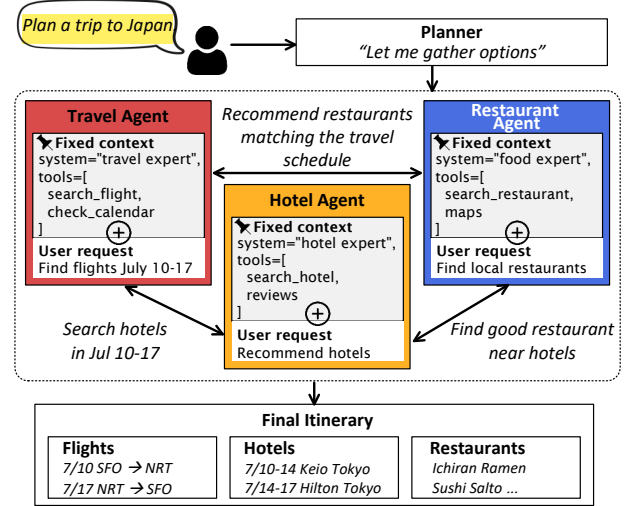
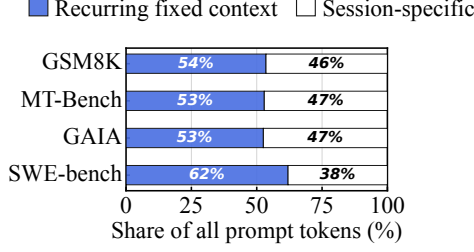


Figure 1: Multi-agent travel planning use case.

specialized reasoning or tool use, and their results are integrated into a final response.

As shown in Figure 1, consider a user asking an AI assistant to plan a trip to Japan. The planner first asks a Travel Agent to search for flights, then a Hotel Agent to recommend places to stay, and finally a Restaurant Agent to find local restaurants before putting everything together. Every time the Travel Agent is invoked, it begins with the same fixed context, including a system prompt such as You are a travel expert. Find the best flight options based on the user’s budget and schedule, together with the same flight-search tool definitions. The Hotel Agent likewise begins with You are a hotel recommendation expert... together with the same hotel-search tools. The task-specific user request is appended after this fixed context. Since this fixed context rarely changes, the KV-cache generated for it can be reused whenever the same agent is invoked, even across different user sessions. As shown in Figure 2, this recurring fixed context accounts for 53–62% of all prompt tokens across the four multi-agent workloads evaluated in this paper, making it a substantial opportunity for KV-cache reuse.

However, today’s LLM serving systems, such as vLLM [16], SGLang [53], and TensorRT-LLM [28], do not fully exploit this opportunity because their cache management is largely reactive, relying on prefix caching and recency-based cache replacement. Returning to our example, after the Travel Agent finishes searching for flights, the planner continues with the Hotel Agent and the



**Figure 2: Share of prompt tokens occupied by recurring fixed context (system prompts, tool definitions, skills, and examples) in four representative multi-agent workloads: GSM8K [5], MT-Bench [52], GAIA [27], and SWE-bench [15].**

Restaurant Agent. As their KV blocks gradually fill the GPU cache, the KV-cache corresponding to the Travel Agent’s fixed context is eventually evicted because it has not been accessed recently. Later, after reviewing the hotel options, the planner decides to compare another flight itinerary and invokes the Travel Agent again. Since its reusable fixed context has already been evicted, the serving system must **recompute the same prefix from scratch**. This pattern repeats throughout a multi-agent workflow. **Although a reusable agent anchor may remain idle while other agents execute, it is likely to be reused when the same agent is invoked again.** Recency-based cache policies **cannot distinguish this temporary idle period from genuinely cold data, leading them to evict precisely the KV blocks that are most valuable to future execution.**

**The problem.** Closing the gap requires the **cache to retain KV blocks that will be used again, instead of simply keeping the blocks that were used most recently.** This is much harder than it appears. Today’s serving engines, such as vLLM [16] and SGLang [53], **manage KV-caches as anonymous blocks indexed by content hashes. They know which block was accessed, but they do not know who used it.** For example, the engine cannot tell whether a block belongs to the Travel Agent, the Hotel Agent, or the Restaurant Agent. **As a result, it cannot tell whether a block is likely to be reused in the near future or can be safely evicted.**

Even if the cache knew which agent produced each block, deciding **what comes next is still difficult.** Modern agent frameworks, such as AutoGen [41] and LangGraph [18], **do not follow a fixed workflow.** Instead, the next agent is chosen by the LLM at runtime. The planner may call the Travel Agent again, switch to another agent, or finish the task altogether. Therefore, future reuse cannot be determined from a predefined workflow. **Recent systems sidestep this difficulty by requiring the workflow to be known in advance,** either as a declared execution graph or developer-provided annotations [12, 21, 30], or by pinning the current session’s KV-cache across tool calls [1, 20]. Neither approach helps when the next agent is decided dynamically at runtime.

Finally, any prediction must be extremely fast. Cache eviction happens whenever GPU memory becomes full and is part of the serving hot path. Learning-based cache replacement policies from the storage literature [3, 33, 38, 43] improve prediction quality through feature extraction and model inference, but spending milliseconds to make a better cache decision would easily cost more than the prefill computation it saves.

**Our solution.** We address this problem by introducing **CACHESCOUT**, a **lightweight KV-cache runtime layer that bridges the gap between agent frameworks and serving engines.** Instead of managing KV blocks solely based on past accesses, **CACHESCOUT observes agent execution, learns how agents are dispatched at runtime, predicts which agent is likely to execute next, and uses this information to guide cache eviction and prefetching.** In this way, cache management becomes aware of future agent reuse rather than merely reacting to previous accesses.

**CACHESCOUT realizes this design through three techniques.** (1) **Transition learner.** It models agent execution as an online first-order Markov chain. For each request, it identifies the agent from the prompt-prefix fingerprint and updates a single transition counter. No offline training or predefined workflow graph is required (Sec. 3.2). (2) **Survival-probability scorer.** It estimates the probability that each cached anchor will be reused from the learned transition matrix and combines this probability with recency and reconstruction cost to rank eviction candidates. As a result, **reusable anchors are retained while execution remains predictable, and the policy naturally falls back toward LRU when it is not (Sec. 3.3).** (3) **Background prefetch.** It predicts the next agent from the same transition matrix and **warms the corresponding anchor between requests, leaving the serving critical path unchanged (Sec. 3.4).** Together, these techniques transform the KV-cache from a passive, LRU-based component into an **active, agent-aware runtime.**

**Summary of results.** We implement **CACHESCOUT** as a lightweight KV-cache runtime layer **on top of vLLM and compare it against vanilla vLLM and Continuum.** We evaluate **CACHESCOUT** on four representative real-world multi-agent workloads using a six-agent supervisor framework with Llama-3.1-8B-Instruct. **CACHESCOUT improves KV-cache hit rate by 10–18 percentage points, reaching 81–85% across all workloads, reduces mean TTFT by 18–45% and mean per-turn latency by 29–38%, and delivers 19–57% higher peak throughput.** Under the same latency budget, it sustains 1.7–12× the load of vanilla vLLM. These gains also extend to larger models. On Qwen3-235B-A22B, a 235B-parameter mixture-of-experts model, **CACHESCOUT reduces mean TTFT by 33–54% on SWE-bench and delivers 37% higher throughput.** We also show that **CACHESCOUT remains robust across different cache capacities and workload conditions while incurring negligible runtime overhead.**

**Contributions.** This paper makes the following contributions.

- We identify a new challenge in multi-agent LLM serving: existing prefix caches are reactive and cannot exploit future agent reuse.
- We propose **CACHESCOUT**, a lightweight KV-cache runtime layer that bridges agent frameworks and serving engines, enabling proactive, agent-aware cache management.
- We design and implement **CACHESCOUT** with three techniques: online transition learning, survival-probability-based eviction, and asynchronous predictive prefetching, while keeping runtime state and overhead low.
- We implement **CACHESCOUT** on top of vLLM and show significant improvements in cache hit rate, TTFT, and throughput across representative multi-agent workloads.

## 2 Motivation and Challenges

### 2.1 Background

**Multi-agent workflows.** Modern LLM applications increasingly organize complex tasks as multi-agent workflows. Instead of relying on a single model invocation, a planner decomposes a user request into a **sequence of specialized agents**, each responsible for a particular task such as **planning, retrieval, coding, or tool use**. These agents may follow a **predefined execution graph** or **be dynamically selected by an LLM at runtime**. Frameworks such as **AutoGen** [41], **LangGraph** [18], **CrewAI** [6], and **OpenAI Swarm** [29] have made this execution model increasingly common in production systems.

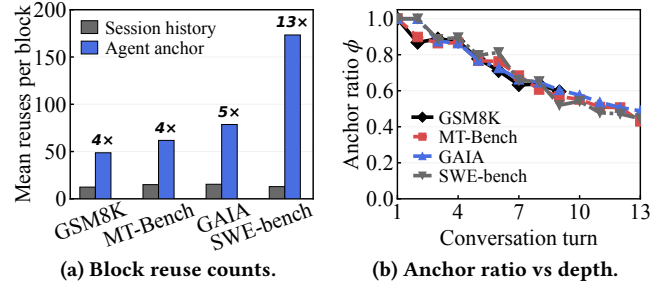
**Agent anchor.** Although different agents perform different tasks, every invocation of the same agent begins with nearly identical prompt prefixes. These prefixes typically consist of a **system prompt** describing the agent’s role, **tool definitions** specifying available APIs, **skill descriptions**, and optionally **few-shot examples**. Throughout this paper, we refer to this reusable prompt prefix as the **agent anchor**. Since an agent anchor **rarely changes across invocations** or user sessions, the **KV-cache generated for it can be reused** whenever the same agent is **executed again**.

**Prefix caching.** Modern serving systems, including **vLLM** [16], **SGLang** [53], **TensorRT-LLM** [28], **LMCache** [22], and **NanoFlow** [54], reduce redundant prefill computation through prefix caching. During the **prefill stage**, the **model computes KV tensors** for every **prompt token**. If a later request shares an **identical prompt prefix**, these **KV tensors can be reused instead of recomputing** the entire prefix. Existing systems differ mainly in how they identify identical prefixes. For example, **vLLM partitions prompts into fixed-size blocks (16 tokens by default)** and identifies reusable KV blocks through block hashing, whereas **SGLang uses RadixAttention**, which organizes prompt prefixes in a **radix tree** to enable token-level prefix matching. Despite these implementation differences, all existing systems identify reusable KV-caches solely according to prompt content.

**Reactive cache management.** Existing **prefix caches** are fundamentally reactive. A **KV block becomes reusable only after it has already been requested and inserted into the cache**. Likewise, cache replacement policies, such as **LRU** [25], **LFU** [2], **ARC** [26], and **LIRS** [14], retain KV blocks solely according to past accesses. The serving engine operates only on content-addressed KV blocks. It neither knows which agent produced a cached block nor whether that agent is likely to be invoked again. As multi-agent workflows become increasingly dynamic, this reactive design leaves many reuse opportunities unexploited.

### 2.2 New Insights

The reactive design above leaves the reuse opportunities of multi-agent workflows **unexploited**. **Recovering them requires the cache to act before reuse happens**, and whether that is possible depends on the structure of multi-agent execution itself. We therefore ask three questions. First, **which cached KV blocks are most valuable to preserve during eviction?** Second, **is future agent execution predictable enough to act on?** Third, **can the execution structure be specified before deployment**, or must it be learned at runtime? We



**Figure 3: Reuse concentration in representative multi-agent workloads. Agent-anchor blocks exhibit substantially higher reuse than session-history blocks, while recurring fixed context remains a significant fraction of each prompt throughout multi-turn interactions.**

answer these questions through a measurement study over representative multi-agent workloads, and the resulting insights directly shape the design of **CACHESCOUT**.

#### 2.2.1 Which KV blocks should be protected?

A natural question is whether all cached KV blocks are equally valuable. If future reuse is uniformly distributed across all KV blocks, conventional cache management policies may already be sufficient. On the other hand, if a small subset of KV blocks contributes disproportionately to future reuse, **selectively protecting those blocks could significantly improve cache efficiency**.

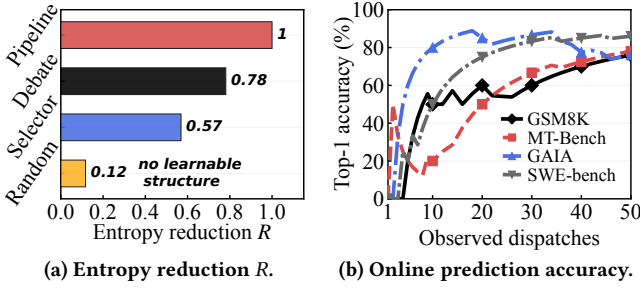
To answer this question, we first measure how often different types of KV blocks are reused. Figure 3a reports the average number of future reuses per KV block across four representative multi-agent workloads. Agent-anchor blocks are reused 49–173 times on average because every invocation of the same agent accesses the same fixed context, both within and across user sessions. In contrast, session-history blocks belong to a single conversation and are reused only 12–15 times on average. Overall, agent anchors receive 4–13× more reuse than session-specific history.

Reuse frequency alone, however, does not determine cache value. We therefore quantify how much of each prompt is occupied by reusable agent anchors. We define the **agent anchor ratio** ( $\phi$ ) as

$$\phi = \frac{\text{system prompts} + \text{tool definitions} + \text{skills} + \text{few-shot}}{\text{total prompt tokens}}. \quad (1)$$

Intuitively,  $\phi$  measures the fraction of each prompt occupied by the reusable agent anchor rather than session-specific conversation history.

Figure 3b plots  $\phi$  over the first half of a median session. Although the reusable fraction gradually decreases as conversation history accumulates, **agent anchors consistently occupy a substantial portion of every prompt**: even a dozen turns into a session, recurring anchor content still accounts for 43–60% of every prompt.



**Figure 4: Execution predictability in representative multi-agent workloads. Execution remains structured across diverse coordination topologies, enabling the online transition learner to quickly achieve high next-agent prediction accuracy.**

**Takeaway.** Agent anchors dominate KV-cache reuse for two reasons. They are reused substantially more frequently than session-specific history and occupy a large fraction of every prompt. Protecting anchor blocks therefore provides the greatest opportunity for improving KV-cache reuse.

### 2.2.2 Is future agent execution predictable?

Having identified which KV blocks are most valuable to preserve, we next ask whether future agent execution is predictable.

Since proactive cache management only requires predicting the next agent invocation, we quantify predictability using the reduction in uncertainty of the next executed agent. Let  $A_t$  denote the agent executed at step  $t$ . We measure the relative reduction in conditional entropy,

$$R = 1 - \frac{H(A_{t+1} | A_t)}{H(A_{t+1})}, \quad (2)$$

where  $R = 0$  indicates that the next agent is independent of the current agent, while  $R = 1$  indicates that the next agent is completely determined by the current one.

Figure 4a reports  $R$  for four representative coordination topologies. Pipeline exhibits deterministic execution ( $R = 1.0$ ) and the moderated Debate topology remains highly structured ( $R = 0.78$ ), while SelectorGroupChat, where the next agent is dynamically selected by an LLM, still achieves  $R = 0.57$ . In other words, conditioning on the current agent eliminates 57% of the uncertainty in the next agent. Only the Random topology shows little learnable structure ( $R = 0.12$ ). Overall, the three practical coordination patterns all exhibit substantial predictability despite making runtime dispatch decisions.

This predictability is also actionable at runtime. Figure 4b replays each workload’s execution trace through an online first-order transition counter, the same statistic CACHESCOUT maintains: predicting the next agent as the most frequent observed successor of the current agent reaches 76–86% top-1 accuracy within 50 observed dispatches, without any offline training.

**Takeaway.** Future agent execution is not random. Although multi-agent workflows are dynamically determined at runtime, they retain strong local transition structure that a lightweight online model can

capture within tens of dispatches, making future agent execution predictable enough to drive proactive cache management.

### 2.2.3 Can execution structure be specified offline?

The previous insight shows that future agent execution is predictable. We finally ask whether this execution structure can be determined before deployment.

To answer this question, we compare the transition matrices of four representative coordination topologies built on the same six-agent framework. Figure 5 shows that execution structure varies dramatically across coordination topologies despite using the same set of agent roles.

Pipeline and Debate are both highly predictable ( $R = 1.0$  and  $R = 0.78$ ), yet their transition matrices are fundamentally different. Pipeline follows a fixed chain through all six agents, whereas Debate concentrates execution on a proposer–challenger exchange between Coder and Reviewer that is periodically adjudicated by a judge. SelectorGroupChat represents the production setting, where the next agent is dynamically selected by an LLM. Although its execution is no longer deterministic ( $R = 0.57$ ), transition probability remains concentrated on a few high-probability edges. In contrast, the Random topology distributes probability almost uniformly across all agent pairs ( $R = 0.12$ ), leaving little exploitable execution structure.

**Takeaway.** Execution structure is not fixed by the agent framework itself. Instead, it is determined by runtime coordination behavior. Consequently, transition models cannot be specified offline and must instead be learned online from runtime execution history.

## 2.3 Challenges

Building an agent-aware KV-cache runtime introduces fundamental challenges. Table 1 summarizes the assumptions underlying existing approaches related to KV-cache management and their limitations for multi-agent serving.

**Challenge #1: Semantic gap between agents and KV blocks.** Existing prefix caches operate on prompt content, not agent semantics. vLLM identifies reusable state through block hashes, while SGLang uses radix-tree prefix matching. These mechanisms can determine whether two prompt prefixes are identical, but they cannot tell which agent produced a KV block, which session it belongs to, or whether the same agent is likely to be invoked again. Prior cache replacement policies, from heuristic approaches such as LRU [25], LFU [2], ARC [26], and LIRS [14] to learning-based methods such as LRB [33], LeCaR [38], GL-Cache [43], and LHD [3], still make decisions over cache objects. They do not expose the agent-level identity needed to protect reusable agent anchors.

**Challenge #2: Dynamic execution without a static graph.** Existing workflow-level optimizations assume that future execution can be described by a declared graph, a fixed pipeline, or developer-provided hints. This assumption breaks for modern agent frameworks such as AutoGen SelectorGroupChat and OpenAI Swarm, where the next agent is often selected by an LLM at runtime. As we show in Sec. 2.2 (Figure 5), even the same six-agent framework can

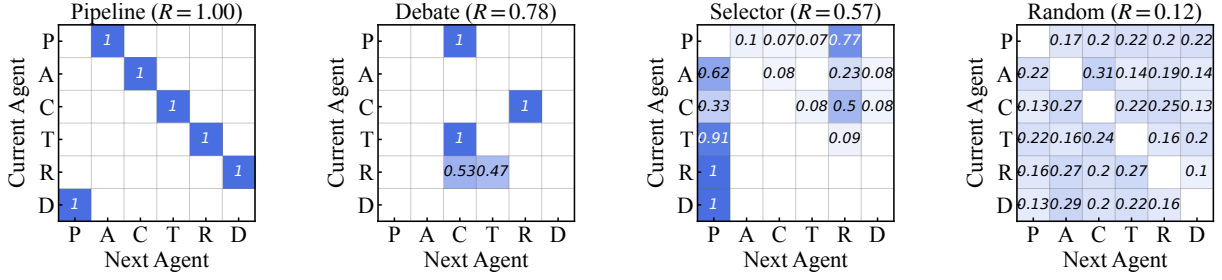


Figure 5: Transition matrices  $P(\text{next} | \text{current})$  learned from four coordination topologies built on the same six-agent framework. The same agents produce dramatically different execution structures under different coordination behaviors, so transition models cannot be specified offline and must be learned at runtime.

Table 1: Comparison of representative approaches to KV-cache management for multi-agent serving. ✓ supported; ✗ not supported.

| Approach (Representative Systems)                                                                 | Cache Abstraction                          | Reuse Policy                                         | Assumption                              | Agent-aware | Learnable Runtime | Limitations for Multi-Agent Serving                                                                                                                   |
|---------------------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------|-----------------------------------------|-------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Reactive serving engines</b> (vLLM [16], SGLang [53], TensorRT-LLM [28])                       | Content-addressed KV blocks                | Prefix matching + recency eviction                   | Recent use predicts reuse               | ✗           | ✗                 | Blind to agent identity; recency-based eviction discards anchors with long reuse distances despite their near-certain reuse.                          |
| <b>Static agent-aware KV management</b> (KVFlow [30], Tokencake [4], Continuum [20], KVCOMM [46]) | KV-cache + static workflow state           | Scheduled prefetch, pinning, or KV adaptation        | Workflow structure known in advance     | ✓           | ✗                 | Static assumptions break under LLM-selected routing; Continuum protects only the current session, and KVCOMM’s approximation degrades output quality. |
| <b>Learning-based cache replacement</b> (LRB [33], LeCaR [38], GL-Cache [43], LHD [3])            | Opaque cache objects                       | Learned reuse prediction                             | Access history predicts reuse           | ✗           | ✓                 | Feature extraction and model inference exceed the serving hot-path budget; no execution semantics.                                                    |
| <b>Agent workflow frameworks</b> (AutoGen [41], LangGraph [18], CrewAI [6])                       | N/A                                        | N/A; delegated to the engine                         | Execution semantics are visible         | ✓           | ✗                 | Sees agent execution but has no visibility into or control over the engine’s KV-cache.                                                                |
| <b>CACHESCOUT (ours)</b>                                                                          | <b>KV-cache + online transition matrix</b> | <b>Transition model guided eviction and prefetch</b> | <b>Execution structure is learnable</b> | ✓           | ✓                 | Gains diminish when execution approaches random routing.                                                                                              |

produce very different transition matrices under different coordination topologies. A cache policy cannot rely on a static workflow graph; it must learn the execution structure from live dispatches.

### Challenge #3: Prediction under serving-system constraints.

Even when future agent execution is predictable, prediction must operate within the stringent latency budget of a production serving engine. Cache eviction is triggered whenever GPU memory becomes constrained and therefore lies directly on the serving critical path. Existing learning-based cache management techniques often improve prediction quality through **complex feature extraction, model inference, or periodic retraining**. While effective for traditional storage caches, these techniques introduce overheads that are difficult to justify in LLM serving, where even microseconds of additional latency accumulate across millions of cache operations. The challenge is therefore not simply to predict future reuse accurately, but to do so using lightweight state, microsecond-scale decision latency, and graceful degradation when execution exhibits little predictable structure.

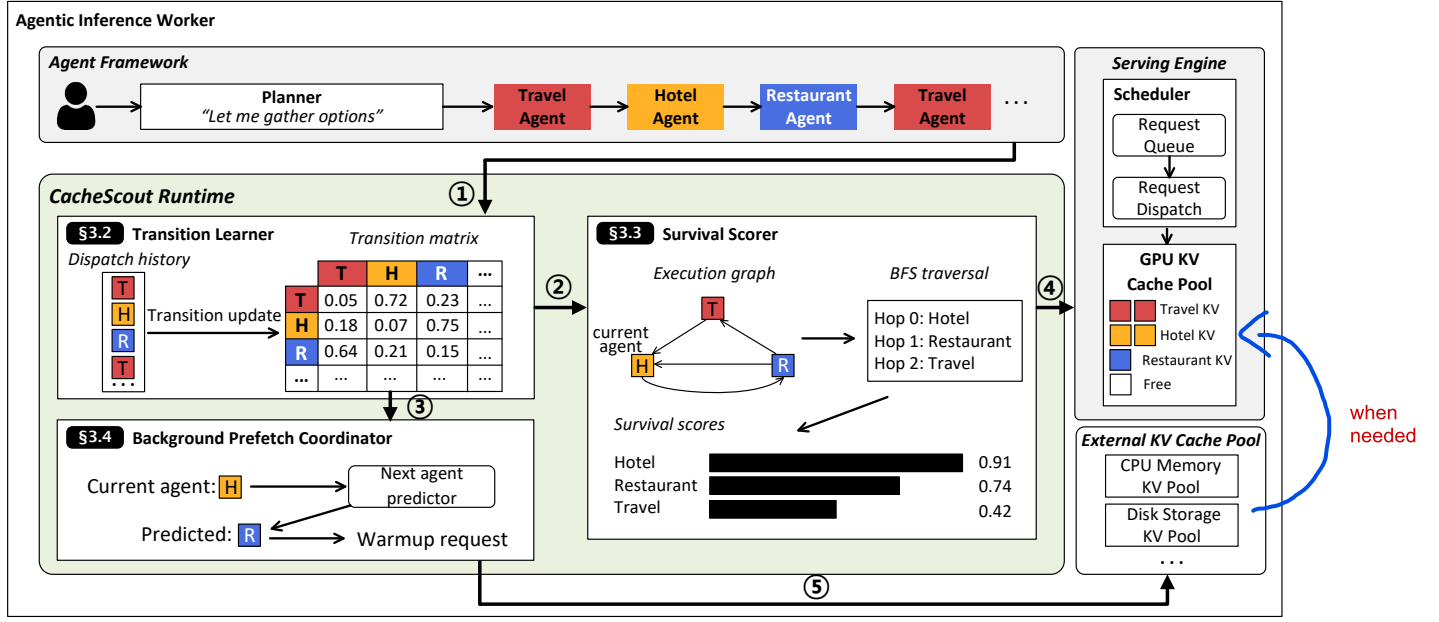
## 3 Design

### 3.1 Runtime Overview

CACHESCOUT is not another cache replacement policy. Instead, it **rethinks the role of the KV-cache runtime in agentic serving**. Existing serving engines efficiently manage KV blocks, but all cache decisions are inherently **reactive**. They **rely solely on past accesses**. Our goal is to answer a different question: *Can the serving system prepare for future execution instead of simply reacting to previous execution?*

Our key idea is to introduce an independent **KV-cache runtime layer between the agent framework and the serving engine**. Rather than embedding agent semantics into the serving engine or pushing cache management into the agent framework, CACHESCOUT separates cache intelligence into its own runtime. This runtime continuously (1) **learns how agents interact**, (2) **predicts which agents are likely to execute next**, and (3) **translates these predictions into cache management decisions**.

Figure 6 illustrates the overall architecture of CACHESCOUT. The runtime resides between the agent framework and the serving



**Figure 6: CACHESCOUT architecture.** CACHESCOUT sits between the agent framework and the serving engine as a lightweight runtime layer. ① Each agent dispatch is observed by the Transition Learner, which updates the online transition matrix. The learned matrix feeds ② the Survival Scorer and ③ the Background Prefetch Coordinator. ④ The scorer converts hop distances on the execution graph into survival scores that guide eviction in the GPU KV-cache pool, and ⑤ the coordinator issues a background warmup request for the predicted agent’s anchor, reloading it from the external KV pools when present.

engine. The agent framework remains responsible for agent scheduling and workflow execution, while the serving engine continues to perform prefix matching, KV allocation, and GPU memory management. CACHESCOUT introduces no changes to either component. Instead, it exchanges lightweight runtime metadata with both sides, allowing execution information to influence cache management without modifying the underlying serving stack.

CACHESCOUT realizes this runtime through three cooperating components. The Transition Learner (Sec. 3.2) is responsible for understanding how a workload executes. As agents interact, it continuously observes runtime dispatches and learns a workload-specific transition model. Instead of assuming a predefined workflow graph, it models execution as an online first-order Markov chain, allowing the runtime to adapt naturally to dynamic routing. The Survival Scorer (Sec. 3.3) answers the next question: which cached state is worth keeping? Rather than asking which block was used most recently, it asks which block is most likely to be reused next. To do so, it estimates the survival probability of each cached agent anchor from the learned transition model and uses this probability to guide cache replacement. The Background Prefetch Coordinator (Sec. 3.4) turns predictions into actions. Once the runtime identifies which agent is likely to execute next, it proactively reconstructs the corresponding high-value KV blocks during idle periods between requests. By moving this work off the serving critical path, CACHESCOUT improves cache readiness without increasing inference latency.

### 3.2 Transition Learner

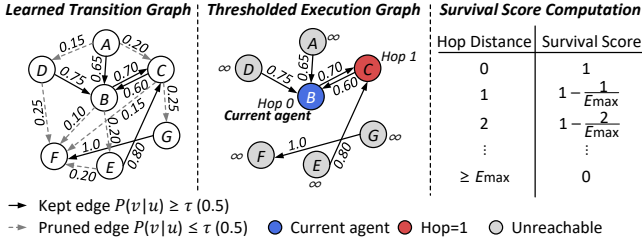
The first task of CACHESCOUT is to infer how a workload executes. This is challenging because modern agent frameworks do not expose a fixed execution graph. The next agent is selected dynamically by LLM reasoning, and the resulting execution structure varies across workloads and evolves over time.

A natural question is how much execution structure needs to be learned. One option is to model complete execution traces using a sophisticated sequence model. However, cache management does not need to predict an entire workflow. It only needs to anticipate the next scheduling decision before cache replacement occurs. Since Sec. 2 shows that local agent transitions already carry substantial predictive signal, CACHESCOUT learns only the local transition structure needed for cache management. To realize this idea, the Transition Learner models agent execution as a first-order Markov chain. Let  $A_t$  denote the agent executed at step  $t$ . For every observed transition  $A_t = i \rightarrow A_{t+1} = j$ , the runtime updates a transition counter  $C_{ij} \leftarrow C_{ij} + 1$ . The transition probability from agent  $i$  to agent  $j$  is then estimated as

$$P_{ij} = P(A_{t+1} = j \mid A_t = i) = \frac{C_{ij} + \epsilon}{\sum_k (C_{ik} + \epsilon)}, \quad (3)$$

where  $\epsilon$  is a small smoothing constant used to avoid zero-probability transitions early in execution. Given the current agent  $A_t = i$ , the predicted distribution over the next agent is simply the  $i$ -th row of the transition matrix:

$$\mathbf{p}_{t+1} = P_{i,:}. \quad (4)$$



**Figure 7: Survival score computation.** CACHESCOUT converts the learned transition graph into a sparse execution graph by pruning low-confidence transitions, then computes survival scores from BFS hop distances.

The most likely next agent is

$$\hat{A}_{t+1} = \arg \max_j P_{ij}. \quad (5)$$

Because each dispatch updates only one counter and one row of the transition matrix, the learner has constant update cost and bounded state. It requires no offline training, no workflow annotations, and no replay of historical traces. The result is a lightweight execution model that evolves with the workload and provides the predictive signal needed by the rest of CACHESCOUT.

### 3.3 Survival Scorer

The second task of CACHESCOUT is not simply to predict which agent will execute next, but to determine which cached agent anchors are worth preserving. A highly probable agent does not necessarily imply that all of its cached blocks should remain in GPU memory, while an infrequently executed agent may still own reusable anchors that are expensive to recompute. Consequently, transition probability cannot be used directly as an eviction policy.

Ideally, we would rank every cached agent anchor according to its probability of being reused within the next  $K$  scheduling steps. We define this quantity as the survival probability.

$$p_{\text{surv}}(a; a_0, K) = \Pr(\exists k \in [1, K] : A_{t+k} = a \mid A_t = a_0), \quad (6)$$

where  $a_0$  is the current agent and  $a$  denotes a candidate agent anchor. Intuitively, the survival probability measures whether an anchor is likely to remain useful in the near future, making it a natural objective for cache replacement.

The exact survival probability is prohibitively expensive to compute, as it requires aggregating transition probabilities over all execution paths within the prediction horizon. Instead, CACHESCOUT approximates survival using the learned execution graph. It first converts the transition matrix into a sparse directed graph by retaining only high-confidence transitions,

$$(a, b) \in E \Leftrightarrow P(b \mid a) \geq \tau, \quad (7)$$

where  $\tau$  is a confidence threshold.

Figure 7 illustrates the approximation process. Starting from the current agent, the runtime performs a single breadth-first search (BFS) over the sparse execution graph to compute the minimum hop distance  $E[a]$  to every reachable agent. Hop distance serves as an efficient proxy for future reuse: anchors expected to be revisited

sooner receive higher survival scores than those farther away in the execution graph. The survival score is then computed as

$$\tilde{p}_{\text{surv}}(a) = 1 - \frac{\min(E[a], E_{\max})}{E_{\max}}, \quad (8)$$

where  $E_{\max}$  bounds the prediction horizon.

Each cached KV block inherits the survival score of its corresponding agent anchor. To improve robustness, CACHESCOUT blends the predicted survival score multiplicatively with a recency signal and weights the result by the block's reconstruction cost,

$$\text{Score}(b) = (\tilde{p}_{\text{surv}}(a_b) + \delta) \cdot (e^{-\lambda \cdot \text{age}(b)} + \delta) \cdot |b|, \quad (9)$$

where  $a_b$  is the anchor associated with block  $b$ ,  $\text{age}(b)$  counts scheduler steps since the block was last accessed,  $\lambda$  is a recency decay rate, and  $|b|$  is the number of tokens the block caches. Blocks with lower scores are evicted first. The product can be read as the expected prefill work lost by evicting  $b$ : the first two factors approximate the probability that the block is reused soon, and  $|b|$  is the recompute cost paid if that reuse misses. Between blocks with equal reuse probability, CACHESCOUT therefore evicts the one that is cheaper to reconstruct. The multiplicative blend lets either signal compensate when the other is weak: under predictable execution, the survival factor protects high-value anchors; when execution becomes less predictable, survival scores flatten and the recency factor dominates, so the policy gracefully degrades toward LRU. The small floor  $\delta$  keeps both factors positive so that newly observed blocks, for which the learner has no signal yet, are not immediately evicted at cold start.

### 3.4 Background Prefetch Coordinator

The final task of CACHESCOUT is to proactively prepare reusable KV-cache before it is requested. While the Transition Learner discovers execution structure and the Survival Scorer determines which cached agent anchors should be retained, neither helps when a future anchor has already been evicted or has not yet been constructed. The Background Prefetch Coordinator closes this loop by proactively warming up high-value agent anchors during idle periods. Algorithm 1 summarizes the overall runtime workflow. A straightforward solution would be to directly insert predicted KV blocks into the cache. However, such a design requires intrusive modifications to the serving engine's memory manager and cache implementation, reducing portability across different inference engines. CACHESCOUT instead adopts a lightweight approach that reuses the engine's existing prefill pipeline. Rather than manipulating KV blocks explicitly, the coordinator issues a lightweight warmup request containing only the predicted agent's reusable anchor, including its system prompt, tool definitions, and a minimal user prompt. Since this warmup follows the engine's normal execution path, the resulting KV blocks are constructed exactly as they would be during a real agent invocation, requiring no modifications to the attention kernels, memory allocator, or cache implementation. Given the current agent  $a_t$ , the coordinator first selects the most likely next agent,

$$a^* = \arg \max_a P(a \mid a_t), \quad (10)$$

and triggers warmup only for the corresponding reusable anchor. Because warmup executes during idle periods between requests,

**Algorithm 1** CACHESCOUT runtime workflow. Numbered steps correspond to the arrows in Figure 6.

**State:** transition counts  $n(\cdot, \cdot)$ ; transition matrix  $\widehat{P}$ ; current agent  $a_t \leftarrow \perp$ ; execution graph  $\mathcal{G}$ ; hop-count table  $E[\cdot]$ ; block-to-agent map  $\text{AGENTOF}[\cdot]$ ; last-access table  $\text{LASTACCESS}[\cdot]$ ; decay rate  $\lambda$ , floor  $\delta$ , threshold  $R_{\min}$ .

```

1: loop
2:    $r \leftarrow \text{framework.NEXTREQUEST}()$   $\triangleright$  agent dispatch
3:    $\text{engine.SERVE}(r)$   $\triangleright$  normal serving path
4:    $\text{BETWEENSTEP}$   $\triangleright$  ③ off critical path
5: end loop

```

During  $\text{SERVE}$ , the block pool raises two events:

```

6: on  $\text{engine.pool.BLOCKTOUCH}(b, s_b)$ :
7:    $\text{OBSERVE TOUCH}(b, s_b)$   $\triangleright$  ① learn transition
8: on  $\text{engine.pool.EVICT}$ :
9:   return  $\arg \min_{b \in \mathcal{B}_{\text{evict}}} \text{SCOREBLOCK}(b)$   $\triangleright$  ④ survival-guided
    eviction

```

```

10: procedure  $\text{OBSERVE TOUCH}(b, s_b)$ 
11:    $a \leftarrow \text{AGENTID}(s_b)$ 
12:    $\text{AGENTOF}[b] \leftarrow a$ ;  $\text{LASTACCESS}[b] \leftarrow \text{NOW}()$ 
13:   if  $a \neq a_t$  then
14:     if  $a_t \neq \perp$  then
15:        $n(a_t, a) \leftarrow n(a_t, a) + 1$ 
16:       update  $\widehat{P}$  from  $n(\cdot, \cdot)$ 
17:     end if
18:      $\mathcal{G} \leftarrow \text{THRESHOLDGRAPH}(\widehat{P}, \tau)$   $\triangleright$  ② refresh scorer state
19:      $E[\cdot] \leftarrow \text{BFS}(\mathcal{G}, a)$ 
20:      $a_t \leftarrow a$ 
21:   end if
22: end procedure

```

```

23: function  $\text{SCOREBLOCK}(b)$ 
24:    $a \leftarrow \text{AGENTOF}[b]$ 
25:    $\text{age} \leftarrow \text{NOW}() - \text{LASTACCESS}[b]$ 
26:   return  $(\tilde{p}_{\text{surv}}(a) + \delta) \cdot (e^{-\lambda \cdot \text{age}} + \delta) \cdot |b|$ 
27: end function

```

```

28: procedure  $\text{BETWEENSTEP}$ 
29:   if  $R(\widehat{P}) \geq R_{\min}$  then
30:      $a^* \leftarrow \arg \max_a \widehat{P}(a | a_t)$ 
31:      $\text{ISSUE WARMUP REQUEST}(a^*)$   $\triangleright$  ⑤ background warmup
32:   end if
33: end procedure

```

all KV construction occurs off the serving critical path, allowing otherwise idle GPU cycles to be converted into future cache hits. Proactive prefetching is beneficial only when execution is sufficiently predictable. CACHESCOUT therefore continuously evaluates the execution predictability metric  $R$  (Eq. 2). When

$$R \geq R_{\min}, \quad (11)$$

the coordinator enables background warmup; otherwise, it disables prefetching and falls back to survival-guided cache replacement alone. This adaptive gating prevents unnecessary GPU work under highly dynamic execution while preserving the benefits of proactive cache management whenever meaningful execution structure exists.

## 4 Implementation

We implement CACHESCOUT as a lightweight runtime layer on top of vLLM v0.11 (V1 engine), consisting of approximately 2,300 lines of runtime logic and an 800-line patch. The runtime exposes four primitives,  $\text{ObserveTouch}()$ ,  $\text{Predict}()$ ,  $\text{ScoreBlock}()$ , and  $\text{Warmup}()$ , that connect execution-aware cache management to the serving engine.

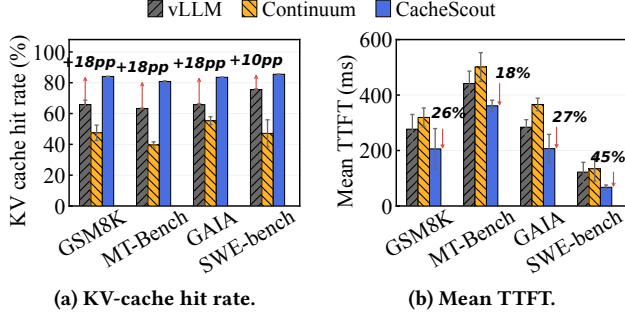
The patch modifies five vLLM source files (the block pool, the engine core, the scheduler, and the scheduler- and worker-side halves of the CPU-offload connector) and introduces three runtime hooks. First, the block pool invokes  $\text{ObserveTouch}()$  at prefix-matching time: each request is fingerprinted from its own prefix block hashes, so a fingerprint change signals an agent dispatch and updates transition statistics and the block-to-agent mapping without any framework cooperation (step ① in Algorithm 1). Second, the eviction path invokes  $\text{ScoreBlock}()$  (Eq. 9) to replace the default LRU ordering with survival-guided ranking, additionally preferring blocks whose contents are already resident in the CPU tier; survival probabilities are computed once per scheduler step and cached. Third,  $\text{Warmup}()$  is executed between agent turns through the standard serving API: after applying the  $R \geq R_{\min}$  gate and a rate limit, it issues an inference request containing only the predicted agent's anchor with  $\text{max\_tokens}=1$ , which the engine recognizes and excludes from transition learning.

CACHESCOUT maintains four runtime data structures: an online transition matrix, a sparse execution graph with shortest-path distances, a hop-distance table, and a block-to-agent mapping, totaling under 25 KB (Sec. 5). Transition counts are updated incrementally on each agent dispatch; the execution graph and hop-distance table are refreshed only when the current agent changes. The eviction hot path performs only a single cached score evaluation (Eq. 9); graph construction and background warmup execute entirely off the critical path.

The implementation reuses vLLM's existing execution pipeline without modifying attention kernels, memory allocators, schedulers, or block hashing. Background warmup is issued as a standard inference request, producing the same KV blocks as a normal invocation. CACHESCOUT can be enabled or disabled through a single environment variable, and reuses vLLM's existing LMCACHE and CPU offloading infrastructure without modification.

## 5 Evaluation

This section evaluates the effectiveness and efficiency of CACHESCOUT on representative multi-agent workloads. Specifically, we seek to answer five questions: (i) Does CACHESCOUT improve KV-cache reuse and end-to-end serving performance compared with existing approaches? (ii) How much do survival-guided eviction and proactive prefetch contribute to the overall performance gains? (iii) Does CACHESCOUT generalize to larger models? (iv) Does CACHESCOUT



**Figure 8: Higher KV-cache hit rates translate into lower inference latency across representative multi-agent workloads.**

remain effective under different cache capacities and serving conditions? (v) What runtime overhead does CACHESCOUT introduce?

## 5.1 Experimental Setup

**Testbed.** We run experiments on a server with eight NVIDIA RTX PRO 6000 Blackwell GPUs, each with 96 GB of memory. We use Llama-3.1-8B-Instruct as the main model and run all systems under the same vLLM configuration, GPU memory budget, decoding parameters, and prefix cache settings. For the model-scale study (Sec. 5.4) we use Qwen3-235B-A22B-FP8, a 235B-parameter mixture-of-experts model, with tensor parallelism across four NVIDIA H200 GPUs each with 141 GB memory and connected via NVLink.

**Workloads.** We evaluate CACHESCOUT on four representative multi-agent workloads: GSM8K [5], MT-Bench [52], GAIA [27], and SWE-bench [15]. All four run on the same six-agent supervisor framework with tools and AutoGen’s SelectorGroupChat for dynamic LLM-selected routing. Together, these workloads cover a broad spectrum of execution characteristics, from short mathematical exchanges (GSM8K) and multi-turn conversation (MT-Bench) to tool-heavy web reasoning (GAIA) and long, prefill-heavy software-engineering sessions (SWE-bench).

**Baselines.** We compare CACHESCOUT against two representative KV-cache management systems. vLLM serves as the production baseline using reactive prefix caching with LRU eviction over block-aligned KV-caches. Continuum represents cross-session KV reuse through TTL-based pinning around tool-call boundaries. Following the observed inter-turn latency in AutoGen, we set the TTL to 0.3 s. The model-scale study (Sec. 5.4) compares against vanilla vLLM only, as Continuum’s vLLM fork does not support this model.

**Metrics.** Our evaluation focuses on the following metrics:

- **KV-cache hit rate:** The fraction of prompt tokens served from the KV-cache, computed as  $\text{total\_cached\_tokens} / \text{total\_prompt\_tokens}$ .
- **Time-to-first-token (TTFT):** The latency from request arrival to the first generated token, measuring user-perceived responsiveness.
- **Per-turn latency:** The end-to-end latency of an individual agent invocation, including both prefill and decoding.
- **Throughput:** The number of completed agent turns per second under a fixed workload and hardware configuration.

## 5.2 Main Results: CACHESCOUT improves KV-cache reuse and serving performance

CACHESCOUT aims to improve both KV-cache reuse and end-to-end serving performance for dynamic multi-agent workloads. We first examine cache effectiveness through KV-cache hit rate, and then evaluate its impact on serving performance.

**KV-cache hit rate and Time-to-first-token.** Figure 8a shows that CACHESCOUT consistently achieves the highest KV-cache hit rate across all four workloads, improving over vanilla vLLM by 10–18 percentage points despite dynamic LLM-selected routing. The higher hit rate translates directly into faster responses: Figure 8b shows that CACHESCOUT reduces mean TTFT by 18–45% across all four workloads. Figure 9 further shows that the improvement holds across the latency distribution: median TTFT drops by 14–52% (e.g., 231→114 ms on GAIA and 239→115 ms on GSM8K), and P99 TTFT drops by 21–52% on three of the four workloads (711→342 ms on SWE-bench), so CACHESCOUT is below vanilla vLLM at the median on every workload. This indicates that the benefit is not limited to a small fraction of requests but consistently improves user-perceived responsiveness.

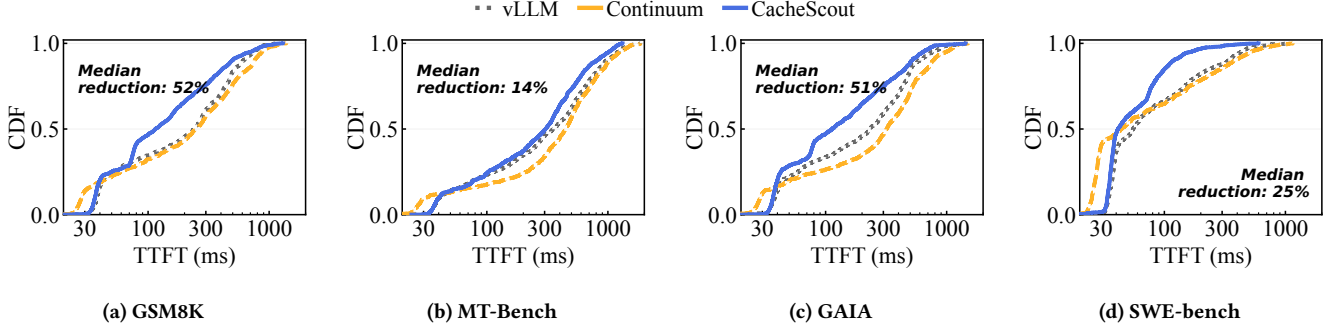
**End-to-end latency and throughput.** The higher KV-cache hit rate translates directly into better serving performance. As shown in Figure 10, CACHESCOUT reduces mean per-turn latency by 29–38% while delivering 19–57% higher peak throughput across all four workloads. Figure 11 shows how this advantage compounds with load: vanilla vLLM’s throughput plateaus once its KV-cache saturates, while CACHESCOUT continues to scale, and within the same mean-latency budget CACHESCOUT sustains 1.7–12× the arrival rate of vanilla vLLM and 4.2–16× that of Continuum. Together, these results demonstrate that improving KV-cache reuse benefits not only request latency but also the overall efficiency of multi-agent serving.

## 5.3 Design Breakdown: Predictive eviction provides the primary performance gains

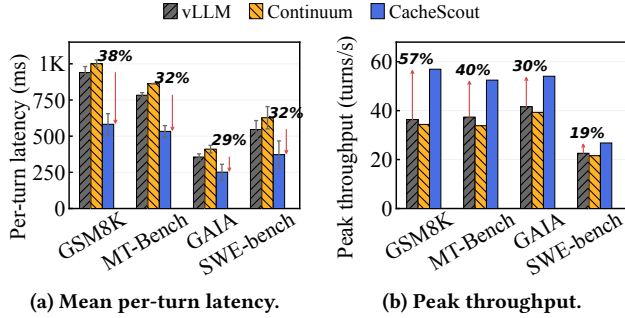
CACHESCOUT combines two predictive runtime mechanisms: predictive eviction and background prefetch. To understand their individual contributions, we evaluate three configurations: vanilla vLLM, eviction only, and the complete CACHESCOUT runtime. Figure 12 summarizes their impact on KV-cache hit rate, mean TTFT, per-turn latency, and throughput.

**Predictive eviction.** Figure 12 shows that predictive eviction provides the dominant source of improvement on the four real-world workloads. Enabling predictive eviction alone increases KV-cache hit rate by 18–22 percentage points over vanilla vLLM, whereas background prefetch alone adds at most one percentage point. The higher cache hit rate directly translates into lower per-turn latency and higher throughput (Figures 12c and 12d), demonstrating that execution-aware cache replacement is the primary contributor to CACHESCOUT’s performance under realistic multi-agent workloads.

**Background prefetch.** The effectiveness of background prefetch depends on what standalone warmup can preserve. On its own, prefetch is inert on these workloads: the median turn carries no warmable agent prefix, and the blocks a warmup recomputes are



**Figure 9: TTFT cumulative distributions across representative multi-agent workloads. CACHESCOUT consistently shifts the distribution toward lower latency.**



**Figure 10: CACHESCOUT consistently achieves lower mean per-turn latency and higher peak throughput across representative multi-agent workloads.**

evicted by LRU before the predicted agent arrives. Combined with predictive eviction, however, warmup re-activates agent anchors that eviction then protects: the complete runtime reduces per-turn latency by a further 28% over eviction alone on GAIA (251 vs. 347 ms) and achieves the best overall serving performance on both serving metrics (Figures 12c and 12d).

These results validate the design of CACHESCOUT. Predictive eviction serves as the primary optimization for dynamic multi-agent workloads, while background prefetch provides additional gains when execution exhibits strong temporal regularity. This complementary behavior motivates CACHESCOUT’s adaptive prefetch policy.

#### 5.4 Scalability: CACHESCOUT remains effective under varying deployment settings and model scale

We evaluate the robustness of CACHESCOUT under varying model scale, GPU cache capacities, and offered load.

**Model scale.** To validate that CACHESCOUT’s mechanisms are not specific to small models, we repeat the serving comparison with Qwen3-235B-A22B-FP8, a 235B-parameter mixture-of-experts model served with tensor parallelism across four H200 GPUs. As shown in Figure 13a, CACHESCOUT’s hit rate advantage transfers unchanged: it improves KV-cache hit rate by 7–13 percentage points across all four workloads. On SWE-bench, whose long prompts keep pre-fill on the costly path, this hit rate advantage compounds across

the entire load range: sweeping the arrival rate from 0.2 to 2.0 sessions/s (Figure 13b–d), CACHESCOUT reduces mean TTFT by 33–54% and mean per-turn latency by 26–36% at every rate, and where vanilla vLLM’s throughput saturates at 5.2 turns/s, CACHESCOUT continues scaling to 7.2 turns/s (+37%). These results indicate that CACHESCOUT generalizes across model scale.

**GPU cache capacity and load.** As shown in Figure 14a, CACHESCOUT maintains a nearly constant KV-cache hit rate of 86–87% as the GPU cache budget varies from 100 to 200 blocks (budgets below the workload’s ~93-block maximum request footprint cannot admit every request and are excluded). In contrast, vanilla vLLM improves gradually from 64.4% to 76.6%, requiring substantially more GPU memory to approach the hit rate achieved by CACHESCOUT. CACHESCOUT is equally robust to load: across the full arrival-rate sweep of Figure 11 (0.2–50 sessions/s), its KV-cache hit rate holds at 86–87% while vanilla vLLM stays near 65–68%. These results demonstrate that CACHESCOUT remains effective under both memory pressure and increasing load.

**Adaptive prefetch gate.** Figure 14b evaluates the adaptive prefetch gate introduced in Sec. 3.4. With the gate enabled, CACHESCOUT maintains stable per-turn latency once sufficient cache capacity is available and degrades gracefully as GPU memory becomes more constrained. In contrast, disabling the gate increases per-turn latency by up to 22% at small cache sizes because unnecessary background warmup competes with foreground inference for GPU resources.

#### 5.5 Runtime Overhead: CACHESCOUT incurs negligible runtime overhead

We evaluate the runtime overhead of CACHESCOUT through in-process microbenchmarks measuring coordinator memory usage and hot-path latency.

**Memory footprint.** As shown in Figure 15a, the coordinator state converges quickly as execution history accumulates and remains below 25 KB even with 24 agents, indicating negligible memory overhead.

**Runtime latency.** Figure 15b shows that both runtime operations incur only microsecond-level overhead. ObserveTouch requires approximately 1  $\mu$ s per block touch, while PredictSurvival completes

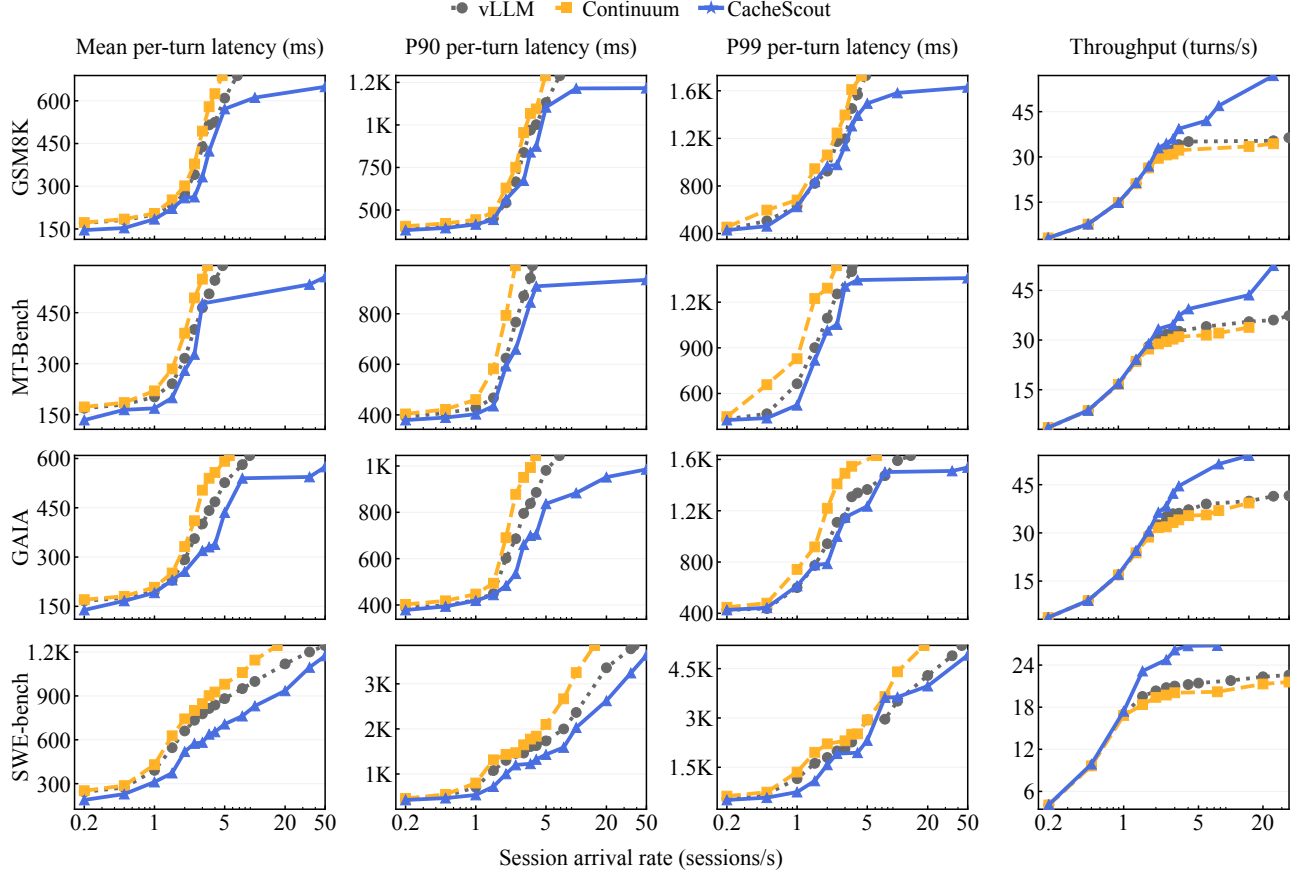


Figure 11: Latency and throughput versus session arrival rate. CACHESCOUT delays cache saturation, sustaining lower latency and higher throughput under increasing load.

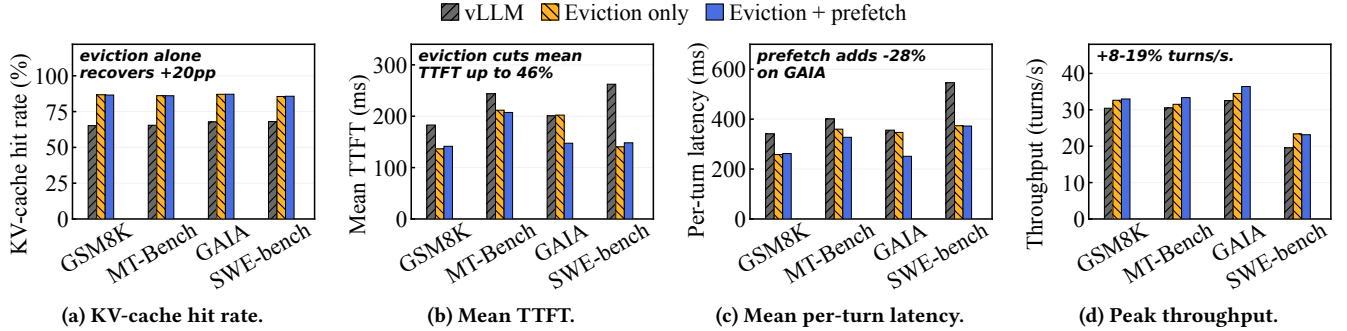


Figure 12: Mechanism ablation. Survival-guided eviction accounts for most of the cache hit rate improvement, while background prefetch further reduces latency and improves throughput.

within 6  $\mu$ s even for 24 agents. These costs are negligible compared with LLM serving latency.

## 6 Related Work

**DAG-aware KV-cache management.** Existing LLM serving engines such as vLLM [16] and SGLang [53] provide efficient prefix caching, but their cache management remains largely reactive. In agentic workflows, this triggers premature eviction of the agent anchor between steps and forces frequent prefill recompute on bytes

the engine just processed. To solve this, a line of work models the agent calling stack as a DAG known a priori, which lets the runtime anticipate future memory accesses along the declared workflow. For example, KVFlow [30] declares the agent step graph via SGLang `sgl.function` annotations and scores eviction by step-distance on the declared graph. SAGA [12] schedules agent workflows atomically using Agent Execution Graphs supplied by the agent framework, and falls back to runtime pattern inference at a 15.6% TCT

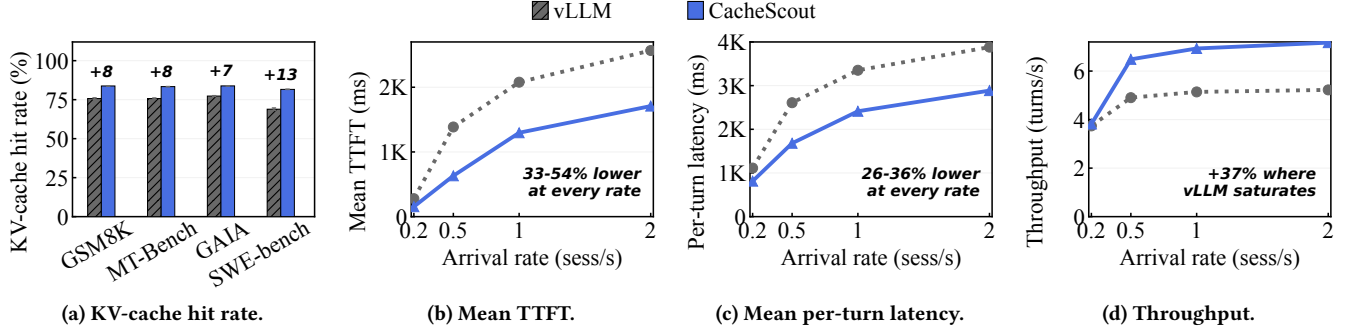


Figure 13: CACHESCOUT on Qwen3-235B-A22B-FP8 (tensor parallelism over four H200 GPUs). KV-cache hit rates are reported across all workloads, while latency and throughput are evaluated on SWE-bench. CACHESCOUT consistently improves cache reuse, reduces inference latency, and sustains higher throughput under increasing load.

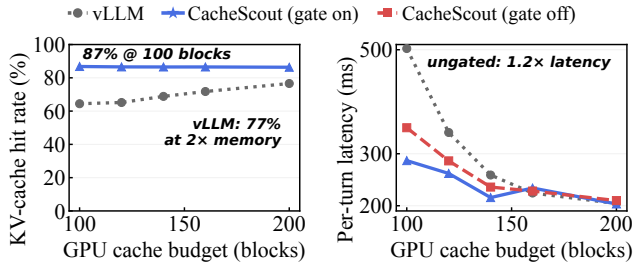


Figure 14: Sensitivity to GPU cache budget. CACHESCOUT maintains high KV-cache hit rates under tight GPU memory budgets, while adaptive prefetch reduces latency without introducing unnecessary warmup overhead.

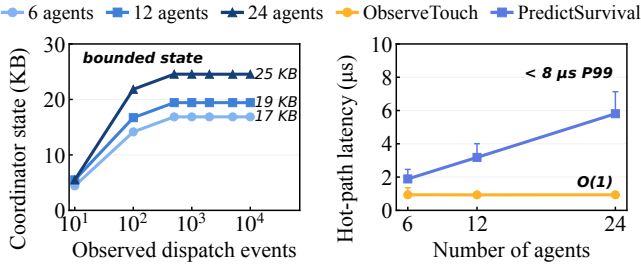


Figure 15: Runtime overhead. CACHESCOUT maintains a compact runtime state and introduces negligible hot-path overhead, even as the number of agents increases.

penalty when framework hints are unavailable. Parrot [21] constructs a request DAG from developer-supplied Semantic Variables that annotate input/output placeholders in prompts, and uses the DAG to colocate dependent requests and share common prompt prefixes across them. PBKV [51] also predicts agent invocations to guide eviction and prefetching, but its neural predictor is trained offline per workload and reuses KV only within a single workflow execution. These systems rely on developer annotations or framework-provided execution graphs, but in real-world agentic

workflows such as a static DAG often cannot be constructed beforehand. CACHESCOUT instead learns the per-workload transition matrix online without any deploy-time-declared graph.

**KV-cache retention across execution stalls.** Agentic workflows interleave LLM generation with external function calls; while a tool runs, the GPU pauses, the session’s KV is evicted under concurrent traffic, and the next step pays a costly re-prefill when the tool returns. One line of work prevents the session’s KV from being evicted during the tool call, either based on the predicted call duration or on the request’s memory footprint over the call’s lifetime [1, 20, 32]. Another line of work fills the tool pause with other agents’ work and reloads the paused agent’s KV from CPU just before the tool returns [4, 24, 39]. These approaches retain only the current agent’s KV state within a session. CACHESCOUT addresses a different and broader problem: it prefetches the next agent’s anchor at the runtime layer before the call arrives, hitting the cache even when the next call dispatches to a different agent or a different session.

**Faster agent serving.** The latency of individual agents is constrained by the LLM’s sequential autoregressive decoding. Unlike chatbot interactions, agentic workflows are inherently structured, and a line of work accelerates serving by exploiting that structure to bypass or parallelize token-by-token generation. Token-level methods bypass autoregression via structured generation, constraining output to a grammar so that deterministic spans need not be sampled token-by-token [7, 40]. Sequence-level methods speculatively execute the agent’s predicted next step in parallel with the LLM’s deliberation [31, 35]. At the framework level, Nalar [17] decouples workflow specification from execution and schedules agent invocations through dependency-carrying futures. Runtime layers have also been proposed to enforce serving policies for agentic workloads [47, 48]. This line of work is complementary and orthogonal to CACHESCOUT, and can be combined with CACHESCOUT to further increase the efficiency of agent inference.

**Other KV-cache optimizations.** Other work optimizes KV-cache management along complementary dimensions, including non-prefix KV-cache sharing with approximation [11, 44], lossy KV-cache reuse with lossless output guarantees [45], joint KV-cache compression and eviction [9], cross-LLM KV-cache reusing with

partial layer recomputation [23], request routing that is aware of KV-cache location [34], and improving paged attention for multi-modality models with different shapes of KV-cache [37]. While this line of work also makes KV-cache in LLM inference more practical in multiple use cases, CACHESCOUT is complementary to them as it targets agent-level reuse across session boundaries, an axis orthogonal to these per-block optimizations.

## 7 Conclusion

CACHESCOUT is an agent-aware KV-cache runtime layer for multi-agent LLM serving that improves cache reuse by leveraging agent execution semantics. By learning execution transitions online, it enables predictive cache management without requiring predefined workflow graphs, offline training, or modifications to the serving critical path. Experiments on representative real-world multi-agent workloads show that CACHESCOUT substantially improves KV-cache hit rate, reduces TTFT and end-to-end latency, and increases serving throughput. CACHESCOUT has been implemented on top of vLLM and will be open-sourced along with the benchmark suite. CACHESCOUT demonstrates that incorporating agent execution semantics into KV-cache management is an effective and practical approach for efficient multi-agent LLM serving.

## References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. Intercept: Efficient intercept support for augmented large language model inference. *arXiv preprint arXiv:2402.01869* (2024).
- [2] Alfred V Aho, Peter J Denning, and Jeffrey D Ullman. 1971. Principles of optimal page replacement. *J. ACM* 18, 1 (1971), 80–93.
- [3] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving cache hit rate by maximizing hit density. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 389–403.
- [4] Zhuohang Bian, Feiyang Wu, Teng Ma, and Youwei Zhuo. 2025. Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. *arXiv preprint arXiv:2510.18586* (2025).
- [5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [6] CrewAI. 2024. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>.
- [7] Yixin Dong, Charlie F Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. 2025. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems* 7 (2025).
- [8] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. 2025. Deepresearch bench: A comprehensive benchmark for deep research agents. *arXiv preprint arXiv:2506.11763* (2025).
- [9] Shaoting Feng, Yuhuan Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, et al. 2025. EVICPRESS: Joint KV-Cache Compression and Eviction for Efficient LLM Serving. *arXiv preprint arXiv:2512.14946* (2025).
- [10] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [11] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
- [12] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. *arXiv preprint arXiv:2605.00528* (2026).
- [13] Yuxuan Huang, Yihang Chen, Haozheng Zhang, Kang Li, Huichi Zhou, Meng Fang, Linyi Yang, Xiaoguang Li, Lifeng Shang, Songcen Xu, et al. 2025. Deep research agents: A systematic examination and roadmap. *arXiv preprint arXiv:2506.18096* (2025).
- [14] Song Jiang and Xiaodong Zhang. 2002. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In *Proceedings of the 2002 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 31–42.
- [15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*.
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [17] Marco Laju, Donghyun Son, Saurabh Agarwal, Nitin Kedia, Myungjin Lee, Jayanth Srinivasa, and Aditya Akella. 2026. Nalar: An agent serving framework. *arXiv preprint arXiv:2601.05109* (2026).
- [18] LangChain. 2024. LangGraph. <https://github.com/langchain-ai/langgraph>.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [20] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live. *arXiv preprint arXiv:2511.02230* (2025).
- [21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [22] Yuhuan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, et al. 2025. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).
- [23] Yuhuan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, et al. 2024. DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. *arXiv preprint arXiv:2411.02820* (2024).
- [24] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [25] Richard L Mattson, Jan Gecsei, Donald R Slutz, and Irving L Traiger. 1970. Evaluation techniques for storage hierarchies. *IBM Systems journal* 9, 2 (1970), 78–117.
- [26] Nimrod Megiddo and Dharmendra S Modha. 2003. {ARC}: A {Self-Tuning}, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*.
- [27] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, Vol. 2024. 9025–9049.
- [28] NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>.
- [29] OpenAI. 2024. Swarm: Educational framework exploring ergonomic, lightweight multi-agent orchestration. <https://github.com/openai/swarm>.
- [30] Zaifeng Pan, AJKUMAR DAHYALAL PATEL, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *Advances in Neural Information Processing Systems* 38 (2025), 126246–126265.
- [31] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. 2025. Sherlock: Reliable and Efficient Agentic Workflow Execution. *arXiv preprint arXiv:2511.00330* (2025).
- [32] Rana Shahout, Cong Liang, Shiji Xin, Qianru Lao, Yong Cui, Minlan Yu, and Michael Mitzenmacher. 2025. Fast inference for augmented large language models. *Advances in Neural Information Processing Systems* 38 (2025), 71562–71591.
- [33] Zhenyu Song, Daniel S Berger, Kai Li, and Wyatt Lloyd. 2020. Learning relaxed belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 529–544.
- [34] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. 2025. Preble: Efficient distributed prompt scheduling for llm serving. In *International conference on learning representations*, Vol. 2025. 37057–37082.
- [35] Yifan Sui, Han Zhao, Rui Ma, Zhiyuan He, Hao Wang, Jianxun Li, and Yuqing Yang. 2026. Parallelizing Tool Execution and LLM Generation for Low-Latency Agent Serving. *arXiv preprint arXiv:2603.18897* (2026).
- [36] Abu Hanif Muhammad Syarubany and Chang Dong Yoo. 2025. PentaRAG: Large-Scale Intelligent Knowledge Retrieval for Enterprise LLM Applications. *arXiv preprint arXiv:2506.21593* (2025).
- [37] Dezhan Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. 2025. VL-cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. In *International Conference on Learning Representations*, Vol. 2025. 219–239.
- [38] Giuseppe Vietri, Liana V Rodriguez, Wendy A Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving cache replacement with ml-based lecar. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*.
- [39] Xinming Wei, Jiahao Zhang, Haoran Li, Jiayu Chen, Haoning Guan, Rui Qu, Maoliang Li, Xiang Chen, and Guojie Luo. 2025. Agent\_xpu: Efficient scheduling

- of agentic llm workloads on heterogeneous soc. *arXiv preprint arXiv:2506.24045* (2025).
- [40] Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702* (2023).
- [41] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *First conference on language modeling*.
- [42] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652.
- [43] Juncheng Yang, Ziming Mao, Yao Yue, and KV Rashmi. 2023. GL-Cache: Group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 115–134.
- [44] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.
- [45] Jiayi Yao, Samuel Shen, Kuntai Du, Shaoting Feng, Dongjoo Seo, Rui Zhang, Yuyang Huang, Yuhao Liu, Shan Lu, and Junchen Jiang. 2026. VeriCache: Turning Lossy KV Cache into Lossless LLM Inference. *arXiv preprint arXiv:2605.17613* (2026).
- [46] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, et al. 2025. Kv-comm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *Advances in Neural Information Processing Systems* 38 (2025), 17882–17928.
- [47] Rui Zhang and Liting Hu. 2025. Enabling Fairness Across Multi-modal and Multi-agent Applications. In *2025 IEEE International Conference on Edge Computing and Communications (EDGE)*. IEEE, 90–92.
- [48] Rui Zhang, Chaeun Kim, and Liting Hu. 2026. A Policy-Driven Runtime Layer for Agentic LLM Serving. *arXiv preprint arXiv:2605.27744* (2026).
- [49] Wenlin Zhang, Xiaopeng Li, Yingyi Zhang, Pengyue Jia, Yichao Wang, Huifeng Guo, Yong Liu, and Xiangyu Zhao. 2025. Deep Research: A Survey of Autonomous Research Agents. *arXiv preprint arXiv:2508.12752* (2025).
- [50] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
- [51] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. *arXiv preprint arXiv:2605.06472* (2026).
- [52] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [54] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2024. NanoFlow: Towards optimal large language model serving throughput. *arXiv preprint arXiv:2408.12757* (2024).